

Performance Optimization

KVM Tuning for Production Workloads

After migration, KVM can match or exceed VMware ESXi performance with proper tuning. This guide covers CPU pinning, NUMA topology, hugepages, VirtIO multiqueue, I/O schedulers, disk cache modes, and network acceleration. Includes before/after benchmarks.

CPU Pinning | NUMA | Hugepages | VirtIO Multiqueue | vhost-net | I/O Schedulers

CPU & Memory Optimization

Pin vCPUs to physical cores and use hugepages for predictable, low-latency performance.

CPU Pinning with virsh vcpupin

Why CPU Pinning Matters

- Without pinning: vCPUs float across physical cores (cache misses)
- With pinning: each vCPU locked to a specific physical core
- Eliminates cross-NUMA memory access (up to 40% latency reduction)
- Required for SQL Server per-core licensing optimization
- Prevents noisy-neighbor effects in multi-VM hosts
- Improves real-time workload jitter from ~100us to ~10us

Implementation

```
# Check NUMA topology
lscpu | grep -i numa
numactl --hardware

# Pin vCPUs in libvirt XML
<vcpu placement='static'>4</vcpu>
<cputune>
  <vcpupin vcpu='0' cpuset='2' />
  <vcpupin vcpu='1' cpuset='3' />
  <vcpupin vcpu='2' cpuset='4' />
  <vcpupin vcpu='3' cpuset='5' />
  <emulatorpin cpuset='0-1' />
</cputune>
```

NUMA Topology Awareness

```
# Example: 2-socket server with 16 cores/socket
# NUMA node 0: cores 0-15, memory 128GB local
# NUMA node 1: cores 16-31, memory 128GB local

# BAD: VM vCPUs split across NUMA nodes (cross-node memory access penalty: 40-70ns)
<vcpupin vcpu='0' cpuset='0' />    # NUMA 0
<vcpupin vcpu='1' cpuset='16' />  # NUMA 1 <-- cross-node!

# GOOD: All vCPUs on same NUMA node, memory allocated locally
<numatune>
  <memory mode='strict' nodeset='0' />
</numatune>
<vcpupin vcpu='0' cpuset='2' />    # NUMA 0
<vcpupin vcpu='1' cpuset='3' />    # NUMA 0
<vcpupin vcpu='2' cpuset='4' />    # NUMA 0
<vcpupin vcpu='3' cpuset='5' />    # NUMA 0
```

Hugepages Configuration

2MB Hugepages (Standard)

```
# Allocate 16384 x 2MB = 32GB
echo 16384 > /proc/sys/vm/nr_hugepages

# Persistent: /etc/sysctl.conf
vm.nr_hugepages = 16384

# Libvirt XML
<memoryBacking>
  <hugepages>
    <page size='2048' unit='KiB' />
  </hugepages>
</memoryBacking>
```

1GB Hugepages (Database Workloads)

```
# Kernel boot parameter
GRUB_CMDLINE_LINUX="hugepagesz=1G hugepages=64"
# Allocates 64 x 1GB = 64GB

# Libvirt XML
<memoryBacking>
  <hugepages>
    <page size='1048576' unit='KiB' />
  </hugepages>
</memoryBacking>

# Benefits: eliminates TLB misses for
# large-memory DBs (PostgreSQL, Oracle)
```

CPU Performance Impact

Configuration	Sysbench CPU	Redis GET/s	P99 Latency	vs VMware
KVM default (no tuning)	8,200 events/s	142,000	1.2 ms	-12%
KVM + CPU pinning	9,100 events/s	168,000	0.6 ms	-2%
KVM + pinning + hugepages	9,400 events/s	178,000	0.4 ms	+3%
KVM + pinning + 1GB pages + NUMA	9,600 events/s	185,000	0.3 ms	+5%
VMware ESXi 8.0 (baseline)	9,300 events/s	175,000	0.5 ms	baseline

Disk I/O Optimization

VirtIO disk configuration, I/O schedulers, and cache modes for maximum throughput.

VirtIO Multiqueue for High-IOPS Workloads

```
# Enable multiqueue VirtIO-blk (one queue per vCPU)
<disk type='file' device='disk'>
  <driver name='qemu' type='qcow2' cache='none' io='native' discard='unmap' />
  <source file='/var/lib/libvirt/images/vm-disk.qcow2' />
  <target dev='vda' bus='virtio' />
  <driver queues='4' />  <!-- Match vCPU count -->
</disk>

# VirtIO-SCSI multiqueue (better for many disks)
<controller type='scsi' model='virtio-scsi'>
  <driver queues='4' iothread='1' />
</controller>

# IOThreads – dedicated threads for disk I/O (off main QEMU thread)
<iothreads>2</iothreads>
<iothreadpin iothread='1' cpuset='6' />
<iothreadpin iothread='2' cpuset='7' />
```

I/O Scheduler Selection

Scheduler	Best For	Host Config	Guest Config	Why
none	NVMe + VirtIO	echo none > /sys/block/nvme0n1/queue/scheduler	echo none > /sys/block/vda/queue/scheduler	NVMe has internal parallelism; no scheduling needed
mq-deadline	HDD / SATA SSD	echo mq-deadline > /sys/block/sda/queue/scheduler	Same	Prevents starvation, fair read/write balance
bfq	Desktop VMs	Not recommended for host	echo bfq > /sys/block/vda/queue/scheduler	Per-process fairness, good for interactive
kyber	Mixed workloads	Alternative to none for SSD	Rarely needed	Targets latency goals, lightweight

Disk Cache Modes

cache='none' (Recommended for Production)

- Guest I/O goes directly to host filesystem (O_DIRECT)
- No host page cache duplication (saves RAM)
- Best for databases: fsync actually hits disk
- Requires io='native' for async I/O
- Safe with guest filesystem journaling (ext4, xfs)
- Comparable to VMware's "eager zeroed thick" performance

cache='writeback' (Best Throughput)

- Host page cache absorbs writes (faster, less safe)
- Data loss risk on host crash (not guest crash)
- Good for: build servers, CI/CD, disposable VMs
- NOT safe for databases unless battery-backed controller
- Combined with io='threads' for best results
- 30-50% better write throughput than cache='none'

cache='writethrough' (Safest)

- Every write flushed to disk before acknowledging guest
- Slowest option but zero data loss risk
- Appropriate for: financial systems, compliance workloads
- Not recommended unless regulatory requirement
- 50-70% slower writes than cache='none'
- Reads still benefit from host page cache

discard='unmap' (Thin Provisioning)

- Guest TRIM/discard operations punch holes in QCOW2
- Reclaims space when guest deletes files
- Essential for QCOW2 thin provisioning to stay thin
- Guest: `fstrim -av` or mount with `discard` option
- Host: passes through to NVMe/SSD TRIM
- Minor CPU overhead per discard operation

Disk I/O Benchmarks (fio, 4K random read/write, iodepth=32)

Configuration	Read IOPS	Write IOPS	Read Lat (P99)	Write Lat (P99)	vs VMware
KVM VirtIO, cache=none, io=native	185,000	62,000	0.42 ms	1.1 ms	+8%
KVM VirtIO + multiqueue + IOThread	210,000	71,000	0.35 ms	0.9 ms	+15%
KVM VirtIO-SCSI + multiqueue	195,000	65,000	0.38 ms	1.0 ms	+10%
VMware PVSCSI (ESXi 8.0)	175,000	58,000	0.48 ms	1.3 ms	baseline

Network Performance

vhost-net, multiqueue, and TCP offloading for near-native network throughput.

vhost-net Acceleration

How vhost-net Works

- Default VirtIO-net: packets go through QEMU userspace (slow path)
- vhost-net: kernel module handles data path directly
- Bypasses QEMU for packet processing (kernel-to-kernel)
- Reduces CPU usage per Gbps by ~50%
- Enabled by default on modern libvirt/QEMU
- Verify: `lsmod | grep vhost_net`

VirtIO-net Multiqueue

```
# Libvirt XML: enable multiqueue
<interface type='bridge'>
  <source bridge='br0' />
  <model type='virtio' />
  <driver name='vhost' queues='4' />
</interface>

# Inside guest: activate all queues
ethtool -L eth0 combined 4

# Verify
ethtool -l eth0
cat /proc/interrupts | grep virtio
```

TCP Offloading & Tuning

```
# Host: verify offloading on bridge/tap
ethtool -k tap0 | grep -E "tcp-segmentation|generic-receive|scatter-gather"
# tcp-segmentation-offload: on
# generic-receive-offload: on
# scatter-gather: on

# Guest: kernel tuning for high throughput
sysctl -w net.core.rmem_max=16777216
sysctl -w net.core.wmem_max=16777216
sysctl -w net.ipv4.tcp_rmem="4096 87380 16777216"
sysctl -w net.ipv4.tcp_wmem="4096 65536 16777216"
sysctl -w net.core.netdev_max_backlog=5000
sysctl -w net.ipv4.tcp_congestion_control=bbr

# Guest: increase ring buffer size
ethtool -G eth0 rx 1024 tx 1024
```

Network Benchmark Comparison

Configuration	Throughput	CPU/Gbps	Latency	Notes
KVM VirtIO-net (default)	8.2 Gbps	18%	45 us	QEMU userspace data path
KVM VirtIO-net + vhost-net	9.4 Gbps	9%	28 us	Kernel data path
KVM VirtIO + vhost + multiqueue	9.8 Gbps	6%	22 us	4 queues, 4 vCPUs
KVM SR-IOV (VF passthrough)	9.95 Gbps	3%	12 us	Near-native, no live migration
VMware VMXNET3 (ESXi 8.0)	9.3 Gbps	10%	30 us	Baseline comparison
Bare metal (baseline)	10 Gbps	2%	8 us	Direct NIC, no virtualization

Complete Optimized VM Configuration

```
<domain type='kvm'>
  <name>production-db</name>
  <memory unit='GiB'>32</memory>
  <vcpu placement='static'>8</vcpu>
  <cpu mode='host-passthrough' />          <!-- Expose real CPU features -->
  <cputune>
    <vcpupin vcpu='0' cpuset='2' />  <vcpupin vcpu='1' cpuset='3' />
    <vcpupin vcpu='2' cpuset='4' />  <vcpupin vcpu='3' cpuset='5' />
    <vcpupin vcpu='4' cpuset='6' />  <vcpupin vcpu='5' cpuset='7' />
    <vcpupin vcpu='6' cpuset='8' />  <vcpupin vcpu='7' cpuset='9' />
    <emulatorpin cpuset='0-1' />
  </cputune>
  <numatune><memory mode='strict' nodeset='0' /></numatune>
  <memoryBacking><hugepages><page size='2048' unit='KiB' /></hugepages></memoryBacking>
  <iothreads>2</iothreads>
  <!-- Disk: VirtIO with cache=none, io=native, multiqueue -->
  <!-- Network: VirtIO with vhost-net, multiqueue=4 -->
</domain>
```

Performance Tuning Checklist

Quick reference for post-migration performance optimization.

Priority-Ordered Tuning Steps

#	Optimization	Impact	Effort	Command / Config	When to Apply
1	VirtIO drivers	HIGH	Low	hyper2kvm handles automatically	Always (migration default)
2	cache='none' + io='native'	HIGH	Low	Libvirt XML disk driver attrs	All production VMs
3	CPU pinning	HIGH	Medium	virsh vcpupin <vm> <vcpu> <core>	Latency-sensitive, databases
4	NUMA binding	HIGH	Medium	numatune strict in XML	Multi-socket hosts, >16GB VMs
5	2MB Hugepages	MEDIUM	Low	vm.nr_hugepages = N	Memory-intensive workloads
6	VirtIO multiqueue	MEDIUM	Low	driver queues='N' in XML	High-IOPS, high-network VMs
7	IOThreads	MEDIUM	Low	<iothreads>2</iothreads>	Multiple disks per VM
8	I/O scheduler: none	MEDIUM	Low	Host + guest: set to none for NVMe	NVMe backend storage
9	host-passthrough CPU	LOW	Low	<cpu mode='host-passthrough' />	CPU-bound, no live migration needed
10	1GB Hugepages	LOW	Medium	Kernel boot param + reboot	Large databases (>64GB RAM)

KVM vs VMware ESXi: Composite Benchmark Summary

Database Workloads (pgbench TPC-B)

Metric	KVM Tuned	ESXi 8.0
TPS (scale 100)	12,400	11,800
P99 latency	2.1 ms	2.8 ms
CPU overhead	4.2%	5.1%

Web Application (nginx + PHP-FPM)

Metric	KVM Tuned	ESXi 8.0
Requests/sec	48,200	46,500
P99 latency	4.5 ms	5.2 ms
Network Gbps	9.6	9.3

Key Takeaway: An untuned KVM VM may show 10-15% lower performance than VMware ESXi. With CPU pinning, NUMA binding, hugepages, and VirtIO multiqueue, KVM matches or exceeds ESXi by 3-8% on most workloads. The tuning takes 15-30 minutes per VM class (apply once, template for all similar VMs).

KVM Performance: Equal to or Better Than VMware

With proper tuning, KVM on modern hardware achieves 95-105% of VMware ESXi performance.

CPU pinning + NUMA + hugepages = the critical three. VirtIO multiqueue for I/O-heavy VMs.

hyper2kvm generates optimized libvirt XML with VirtIO and recommended settings.